



Politechnika Wrocławska

Struktury danych

Analiza porównawcza wydajności list wiązanych i tablic dynamicznych w języku C++

Piotr Frąc, Kornel Pawłowski

Marzec 2026

1 Wprowadzenie

Projekt skupia się na analizie porównawczej dwóch fundamentalnych struktur liniowych: *linked list* i *dynamic array*

Celem pracy jest implementacja obu struktur w języku C++ oraz przeprowadzenie testów wydajnościowych dla podstawowych operacji, takich jak:

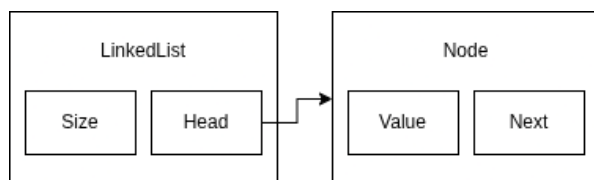
- dodawanie i usuwanie elementów (na początku, na końcu oraz na zadanym indeksie),
- wyszukiwanie elementów o określonej wartości,
- dostęp do elementu poprzez indeks.

Badanie ma na celu wykazanie różnic w złożoności obliczeniowej oraz praktycznym czasie wykonania, wynikających ze specyfiki alokacji pamięci. Pełny kod źródłowy projektu wraz z historią zmian dostępny jest w repozytorium pod adresem: https://github.com/Elrior-programmer/Struktury-Danych/tree/main/P1%20-%20Lista_wizana

1.1 Zaimplementowane struktury danych

Wszystkie struktury danych zostały wykonane zgodnie z wykładami i prezentacjami dr inż. Radosława Rudego, które są dostępne pod linkiem: <https://kam.pwr.edu.pl/jaroslawn-rudy/#tab-struktury-danych-wyklad>

Lista wiązana



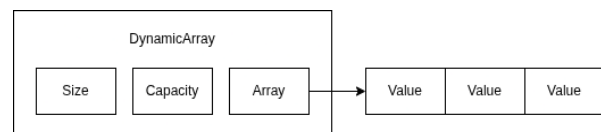
Zaimplementowana **lista** jest przykładem *Listy jednokierunkowej liniowej (Singly Linked List)*, która charakteryzuje się:

- **Jednokierunkowość:** Każdy węzeł (*node*) przechowuje wskaźnik wyłącznie do swojego następcy. Oznacza to,

że przeglądanie struktury możliwe jest tylko od głowy (*head*) w stronę końca listy.

- **Liniiowość (brak cyliczności):** Struktura posiada wyraźnie zdefiniowany początek oraz koniec. Ostatni element przechowuje wartość pustą (`nullptr`), co stanowi jednoznacznie zakończenie sekwencji danych.

Tablica Dynamiczna



Zaimplementowana **tablica dynamiczna** charakteryzuje się specyficznymi usprawnieniami implementacyjnymi:

- **Inicjalizacja niezerowa:** Tablica rozpoczyna pracę z domyślną pojemnością *capacity* = 4, co eliminuje potrzebę realokacji przy pierwszych operacjach dodawania.
- **Progresja geometryczna ($2 \times capacity$):** W przypadku przepełnienia bufora, jego rozmiar jest zwiększany dwukrotnie. Zapewnia to zamortyzowany koszt dopisania elementu na poziomie $O(1)$.
- **Redukcja pamięci ($capacity/2$):** Gdy liczba elementów spadnie do poziomu $size \leq capacity/4$, rozmiar tablicy jest zmniejszany o połowę.

2 Metodyka i środowisko badawcze

W tej sekcji opisano parametry techniczne stanowisk testowych oraz metodologię przeprowadzenia eksperymentów wydajnościowych.

2.1 Narzędzia pomiarowe

Do precyzyjnego wyznaczenia czasu trwania operacji wykorzystano bibliotekę `<chrono>`

oraz zegar o wysokiej rozdzielczości `std::chrono::high_resolution_clock`. Wybór ten pozwolił na rejestrację różnic czasowych rzędu nanosekund, co jest kluczowe przy analizie struktur danych dla małych wartości n .

2.2 Specyfikacja stanowiska badawczego

Do przeprowadzenia testów porównawczych wykorzystano starszą maszynę pomiarową.

Stanowisko (Starsza architektura):

- **Procesor:** Intel Core i5-620
- **Słowo Procesorowe:** 64 Bity
- **RAM:** 8,0 GB DDR3 (1600 MT/s)
- **OS:** Kali GNU/Linux 2026.1
- **Kompilator:** g++ v15.2.0-15

2.3 Dane

W badaniach wykorzystano **Word-aligned (8 B)**: Typ `long long` rozmiar odpowiadający natywnemu słowu architektury x86-64. Dane były tworzone losowo aby uniknąć optymalizacji przez CPU.

3 Wyniki i wnioski

Poniżej zestawiono wyniki pomiarów dla obu struktur według rodzaju operacji. Każda para wykresów przedstawia tę samą operację wykonaną odpowiednio na tablicy dynamicznej i liście wiązanej, co umożliwia bezpośrednie porównanie.

2.4 Wykonane eksperymenty

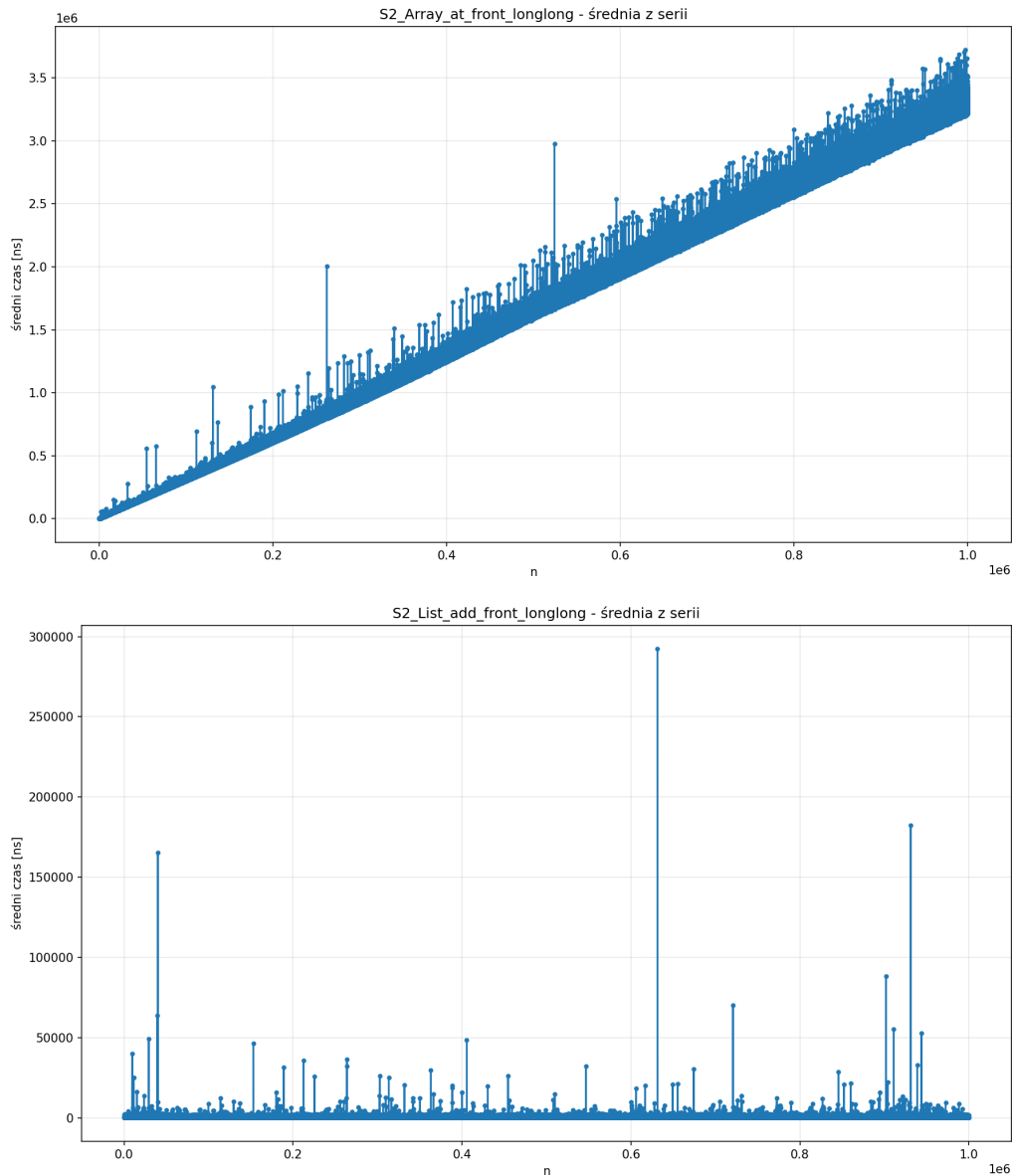
Dla każdej struktury danych przeprowadzono trzy rodzaje eksperymentów, powtarzając je dla wszystkich czterech typów danych (`char`, `long long`, Medium 24B, Large 256B).

- **Dodawanie elementów** dla trzech wariantów wstawiania: na początku struktury, na środku (indeks $n/2$) oraz na końcu.
- **Usuwanie elementów** analogicznie do dodawania, usuwanie badano w trzech miejscach: z początku, ze środka (indeks $n/2$) oraz z końca struktury.
- **Wyszukiwanie pierwszego elementu k** w strukturze o wielkości miliona.

Metodyka pomiarów

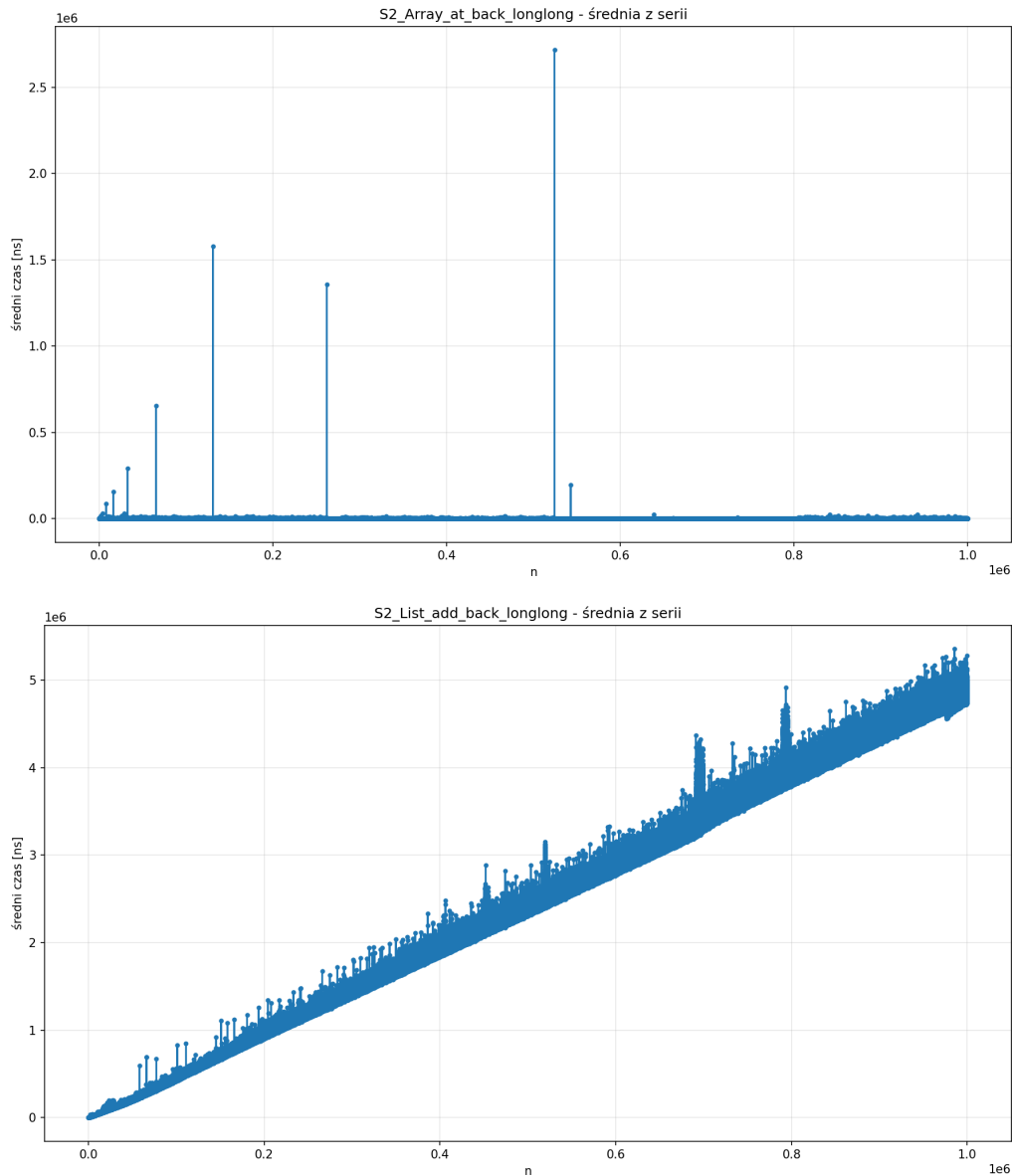
Każdy eksperyment powtórzono 10-krotnie, a wynik końcowy stanowi średnią arytmetyczną z tych prób. Pozwala to zminimalizować wpływ zakłóceń systemowych. Pomiarów wykonywano niezależnie na obu stanowiskach badawczych opisanych w sekcji 2.2.

Dodawanie elementu na początku



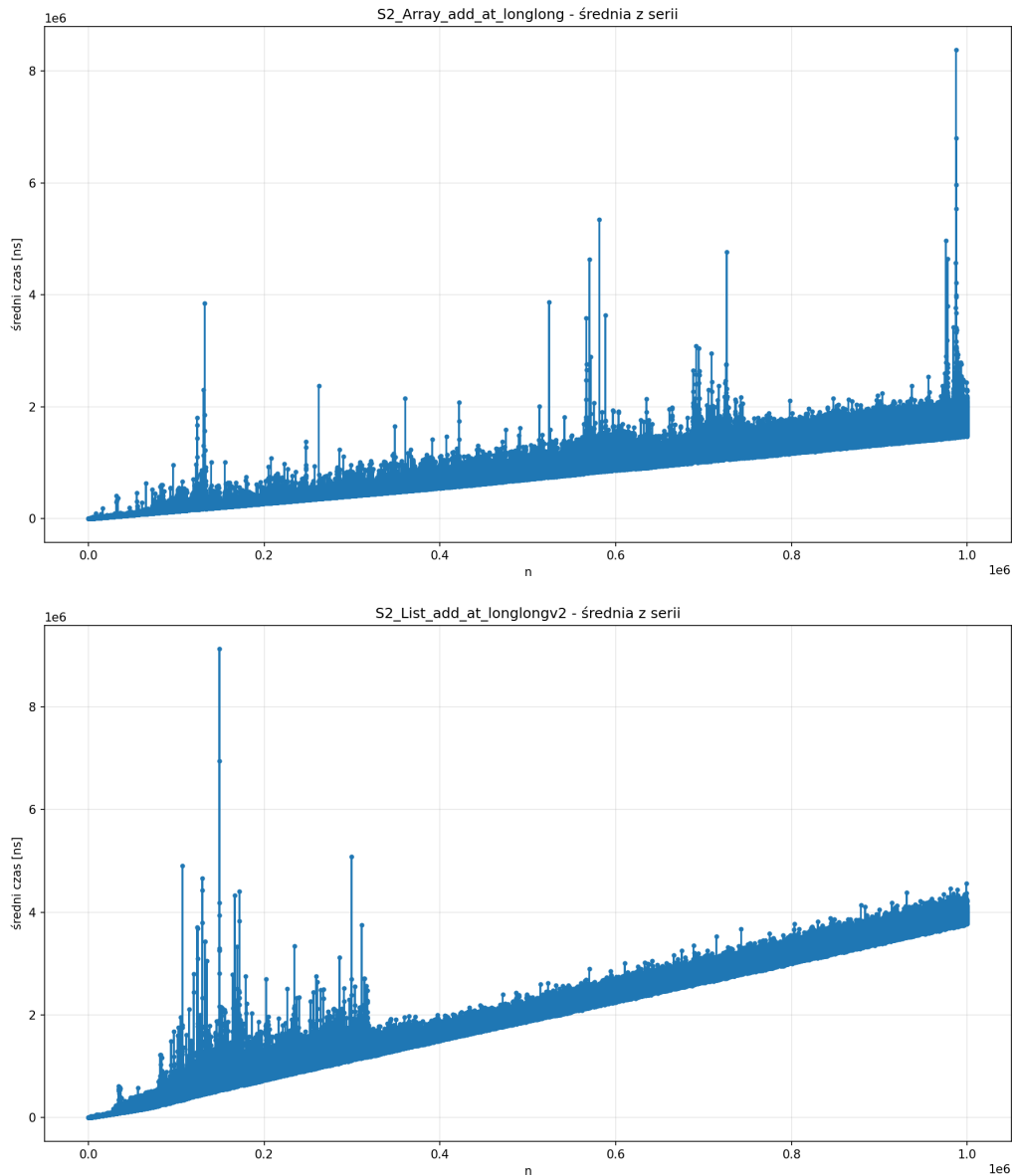
Wnioski: Dodawanie na początku tablicy dynamicznej wymaga przesunięcia wszystkich istniejących elementów o jedną pozycję w prawo, co skutkuje złożonością $O(n)$. Lista wiązana wykonuje tę operację w czasie stałym $O(1)$ — wystarczy utworzyć nowy węzeł i ustawić jego wskaźnik na dotychczasową głowę. Wykresy potwierdzają liniowy wzrost czasu dla tablicy przy rosnącym n oraz niemal poziomą charakterystykę listy.

Dodawanie elementu na końcu



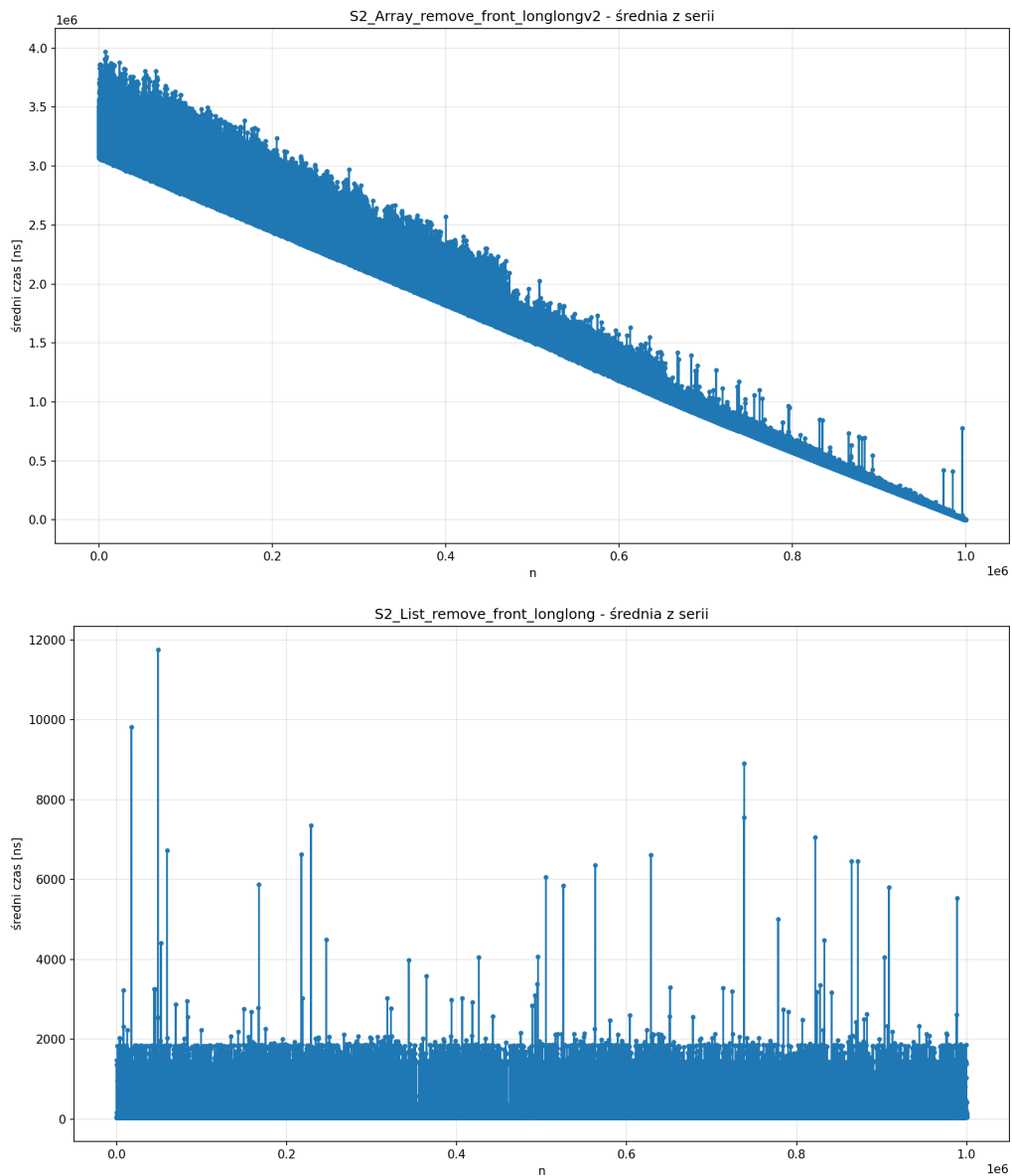
Wnioski: Tablica dynamiczna dodaje element na końcu w zamortyzowanym czasie $O(1)$ dzięki strategii podwajania pojemności. Widoczne skokowe wzrosty czasu odpowiadają momentom realokacji bufora. Lista wiązana nie posiada wskaźnika na ogon (*tail*), więc każde dodanie na końcu wymaga przejścia przez wszystkie węzły — złożoność $O(n)$. Wykres listy wykazuje wyraźny liniowy wzrost czasu wraz z n , podczas gdy tablica utrzymuje niemal stały czas wykonania..

Dodawanie elementu na zadanym indeksie (środek)



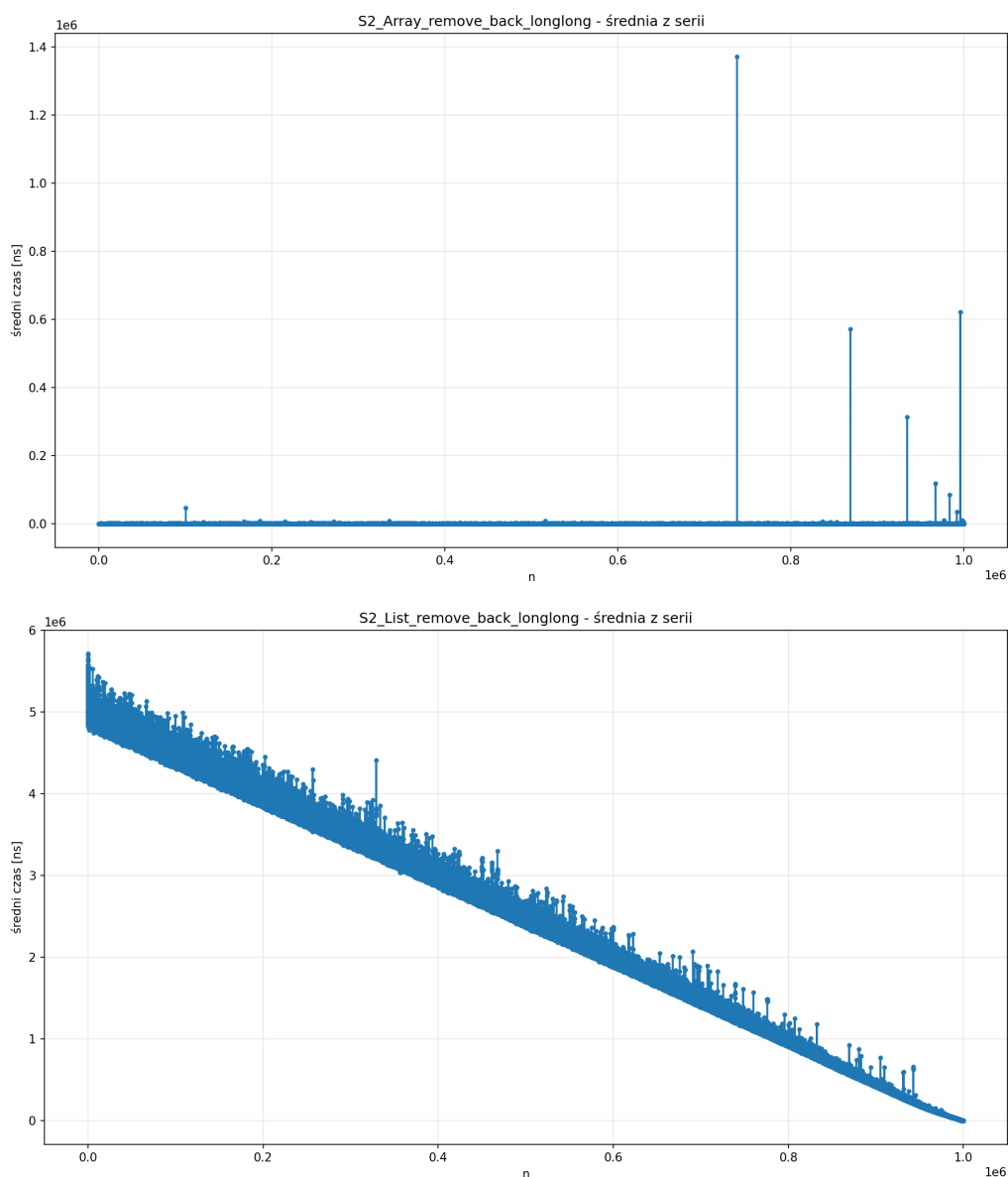
Wnioski: Obie struktury wykazują złożoność $O(n)$ przy wstawianiu w środku — tablica przesuwa elementy, lista musi dotrzeć do węzła o indeksie $n/2$. W praktyce tablica osiąga lepsze wyniki dzięki spójności pamięci podręcznej (*cache locality*): przesuwanie ciągłego bloku danych jest znacznie szybsze niż sekwencyjne dereferencjonowanie wskaźników w liście.

Usuwanie elementu z początku



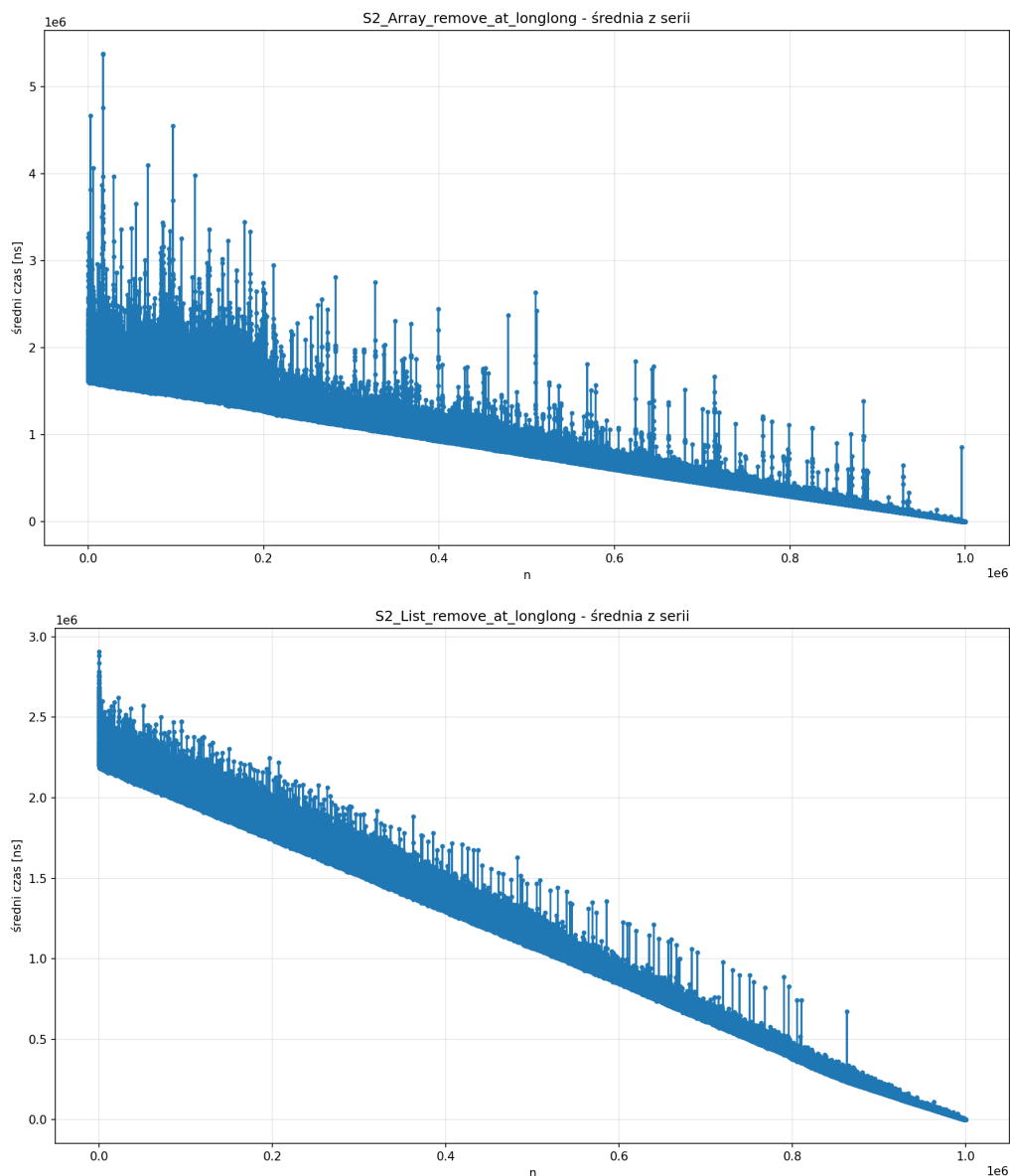
Wnioski: Analogicznie do dodawania: usunięcie pierwszego elementu tablicy wymaga przesunięcia całej reszty ($O(n)$), natomiast lista aktualizuje jedynie wskaźnik głowy ($O(1)$). Różnica wyraźnie rośnie wraz z n , co wykresy potwierdzają rosnącą rozbieżnością krzywych.

Usuwanie elementu z końca



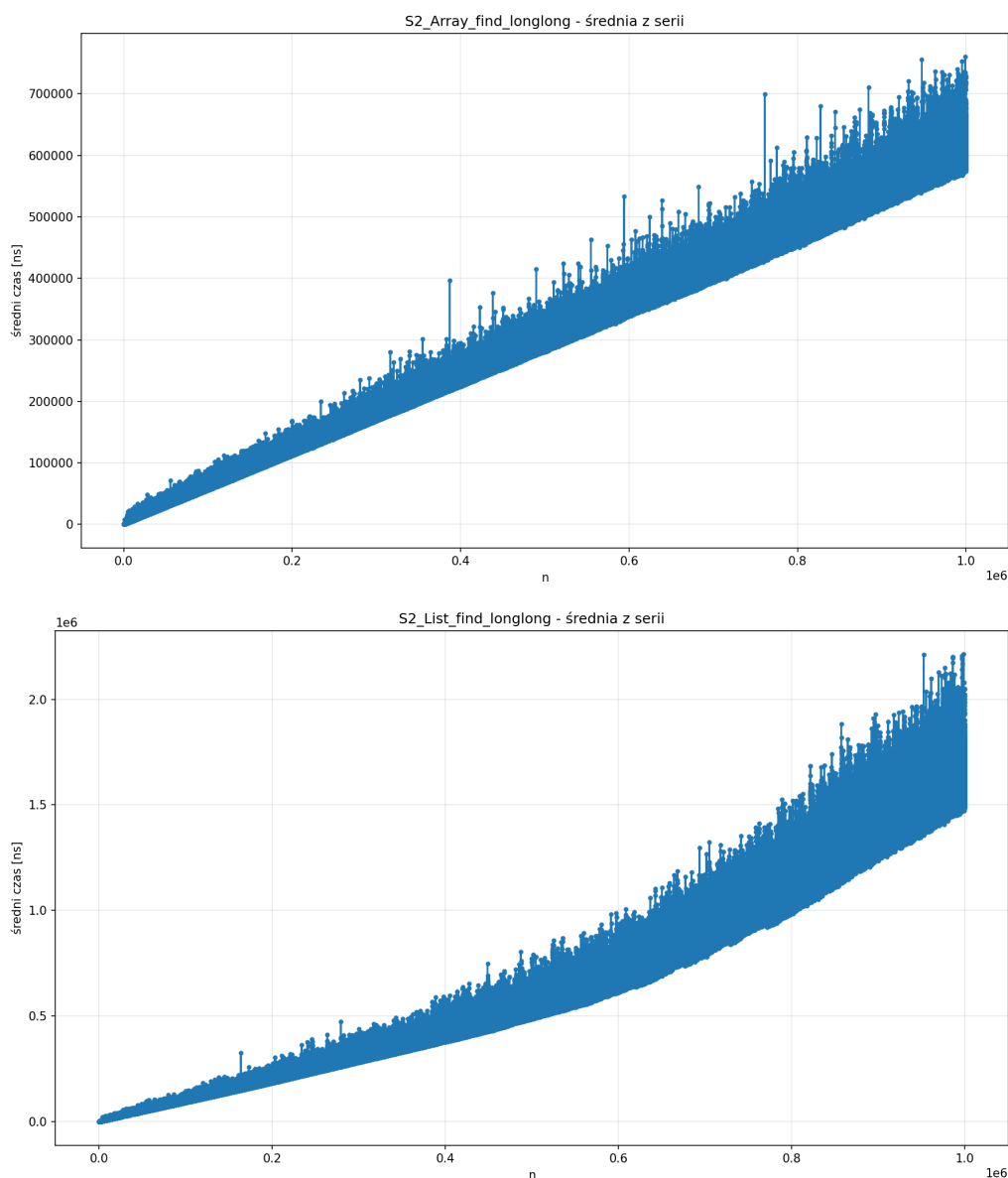
Wnioski: Tablica usuwa ostatni element w czasie $O(1)$ (z ewentualną redukcją pojemności przy $size \leq capacity/4$). Lista wiązana bez wskaźnika na ogon musi każdorazowo przejść całą strukturę, aby dotrzeć do przedostatniego węzła — złożoność $O(n)$. To jeden z kluczowych przypadków, gdzie tablica dynamiczna wyraźnie przewyższa listę jednokierunkową, co potwierdzają wykresy pokazujące liniowy wzrost czasu dla listy.

Usuwanie elementu o zadanym indeksie (środek)



Wnioski: Obie struktury wymagają $O(n)$ — tablica przesuwa elementy, lista przechodzi do węzła poprzedzającego usuwany. Podobnie jak przy wstawianiu w środku, lepsza lokalność pamięci tablicy przekłada się na niższe bezwzględne czasy wykonania.

Wyszukiwanie elementu



Wnioski: Obie struktury realizują wyszukiwanie liniowe $O(n)$. Tablica osiąga znacznie krótsze czasy dzięki ciągłemu układowi danych w pamięci, który pozwala procesorowi efektywnie prefetchować kolejne elementy do cache. Lista wiązana, operując na rozproszonych węzłach, generuje częste chybenia pamięci podręcznej (*cache miss*), co istotnie spowalnia operację dla dużych n .

Podsumowanie

Przeprowadzone eksperymenty potwierdzają teoretyczne złożoności obliczeniowe obu struktur. Lista wiązana dominuje przy operacjach na początku struktury ($O(1)$ vs $O(n)$), natomiast tablica dynamiczna okazuje się lepsza w operacjach na końcu oraz we wszystkich scenariuszach wymagających przeszukiwania, dzięki lepszej lokalności pamięci podręcznej. Wybór struktury powinien być podyktowany profilem dominujących operacji w danym zastosowaniu.